Enhancing Text-to-SQL with Schema-Grounded Symbolic Intermediate Representations

Atharv Kulkarni
athkulk@unc.cs.edu
UNC Department of Computer Science
Project Code

Abstract

Text-to-SQL systems often struggle to follow database schemas, leading to invalid SQL, execution failures, and incorrect answers. This challenge is especially pronounced for smaller language models that must generalize across unseen databases and multi-table schemas. We present a neuro-symbolic approach that introduces a Schema-Grounded Symbolic Intermediate Representation (SIR) between natural language input and SQL generation. In our method, a language model first produces an explicit symbolic plan over schema elements such as tables, joins, columns, filters, grouping, and ordering, and then generates SQL conditioned on that plan. We further train this pipeline with Group Relative Policy Optimization using rewards based on symbolic validity, schema adherence, SQL executability, and execution correctness. Experiments on the Spider benchmark show that the proposed approach improves schema grounding, execution robustness, and downstream text-to-SQL performance, while also making intermediate reasoning more interpretable.

1 Method

We study text-to-SQL on the Spider benchmark, which is designed to evaluate generalization across unseen databases and complex multi-table queries Yu et al. [2018]. Given a database schema and a natural language question as input, the baseline directly generates SQL in a single neural step. In contrast, our approach introduces a Schema-Grounded Symbolic Intermediate Representation (SIR) between the question and the final SQL query. The SIR explicitly represents schema-relevant structure, including selected tables, joins, columns, filters, grouping, and ordering. The model therefore operates in two stages: (1) schema + question $\rightarrow$ SIR, and (2) schema + SIR $\rightarrow$ SQL.

For training, we use Group Relative Policy Optimization (GRPO), following prior work on reinforcement learning for language models Shao et al. [2024]. The model is rewarded for producing a parseable and schema-valid SIR, generating schema-adherent and executable SQL, and returning the correct execution result. This converts text-to-SQL from a purely neural sequence generation problem into a neuro-symbolic pipeline in which neural generation is guided by explicit symbolic structure and execution-based feedback.

2 Experimental Setup

Task and Dataset. We evaluate our approach on Spider Yu et al. [2018], a cross-domain text-to-SQL benchmark designed to test generalization to unseen database schemas and complex multi-table queries. Each example consists of a database schema, a natural language question, and a gold SQL query. We train on the Spider `train` split and evaluate on the

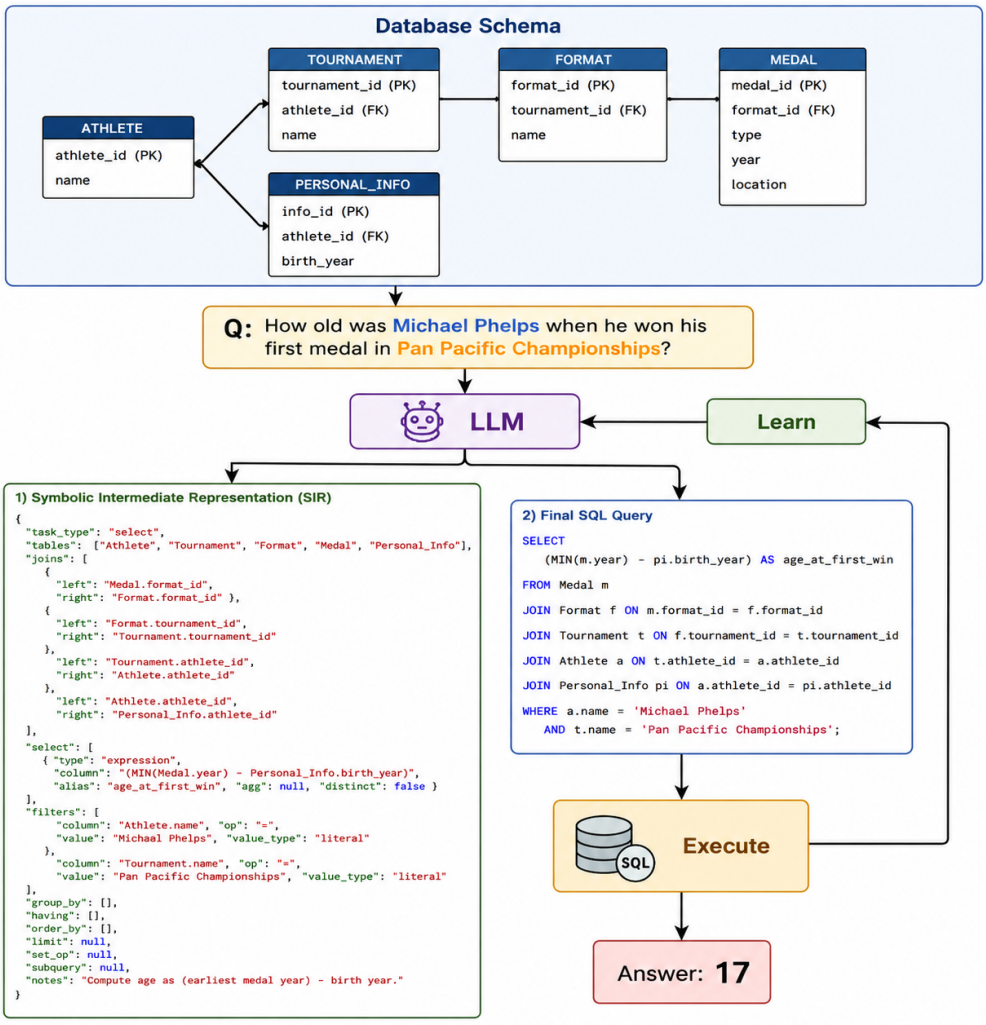

Figure 1: Overview of our neuro-symbolic text-to-SQL pipeline. Given a schema and a natural language question, the model first produces a Schema-Grounded Symbolic Intermediate Representation (SIR) and then generates the final SQL query conditioned on that symbolic plan.

`validation` split. All predicted SQL queries are executed against the corresponding SQLite database.

Input and Output Format. We consider two generation settings. In the **base** setting, the model receives the schema and question and directly generates a final SQL query. In the **SIR** setting, the model is prompted to first generate a *Schema-Grounded Symbolic Intermediate Representation* (SIR) in JSON format and then generate a final SQL query conditioned on that symbolic plan. The SIR explicitly represents the schema-relevant structure of the query, including tables, joins, selected columns, filters, grouping, ordering, and optional aggregation. This changes text-to-SQL from a single-step neural generation problem into a two-stage neuro-symbolic pipeline.

Model and Training Procedure. Our base model is Qwen3-4B. We fine-tune it with LoRA adapters and optimize it using Group Relative Policy Optimization (GRPO) Shao et al. [2024]. For each training prompt, the policy samples 4 rollouts. Each rollout contains both an SIR and a final SQL query. These sampled completions are scored with a structured reward function, and GRPO updates the model according to the relative quality of the sampled

outputs within each group. In our setup, training uses batch size 1, gradient accumulation of 2, and checkpoints are evaluated after **1 Epoch** to **4 Epochs** of training.

Compared Systems. We compare four main systems. The first is the **base model with single-sample decoding**, which directly maps the schema and question to SQL without using SIR and without any training. The second is a **Best-of-N zero-shot baseline** built on the same base model, where no SIR and no training are used, but the model generates $N = 8$ candidate SQL outputs and the best candidate is selected using execution-based correctness. The third is the **SIR model without training**, which introduces the two-stage schema + question $\rightarrow$ SIR $\rightarrow$ SQL pipeline but uses only a single generation and no GRPO optimization. The fourth is the **SIR + GRPO model**, which uses the same two-stage neuro-symbolic pipeline and is evaluated after multiple epochs of GRPO training. This comparison lets us distinguish among three separate effects: the impact of introducing an explicit symbolic intermediate representation, the impact of stronger multi-sample zero-shot decoding, and the impact of reinforcement learning over the SIR-based pipeline.

Best-of-N Zero-Shot Baseline. The Best-of-N baseline is included to make the zero-shot comparison stronger and fairer. GRPO training optimizes over multiple sampled rollouts per prompt, so improvements from the trained system could otherwise be attributed simply to sampling diversity. To control for this, we evaluate an untrained baseline that generates $N = 8$ candidates for each example and selects the best one according to execution-based correctness. This baseline does not use SIR and does not update the model parameters. It therefore isolates the benefit of multi-sample decoding alone, separate from the effect of symbolic intermediate reasoning and separate from the effect of GRPO training.

Interpretation of Training Configurations. In the results tables, the **Training** column describes whether the model has been optimized with GRPO and, if so, how far training has progressed. A mark $\times$ indicates that no GRPO training is applied. The label **1 Epoch** refers to the SIR + GRPO model evaluated after one full pass over the training data. The label **2 Epochs** refers to the same training run evaluated after two passes over the training data. References to **later epochs** denote checkpoints from the same SIR + GRPO training run taken at later stages of training.

Evaluation Metrics. We report both task-level and neuro-symbolic metrics. **Execution Accuracy** measures whether the predicted SQL returns the same final result as the gold SQL. **Execution Success Rate** measures whether the predicted SQL executes without error. **SQL Schema Valid Rate** measures whether the predicted SQL only uses valid tables and columns from the given schema. To analyze schema grounding more directly, we report **Schema-Oriented Table Match Rate** and **Schema-Oriented Column Exact Rate**, which compare the predicted SQL with the gold SQL in terms of selected tables and columns. For the symbolic stage, we report **SIR Parse OK Rate**, which measures whether the generated SIR is well-formed and parseable, **SIR Valid Rate**, which measures whether it uses only valid schema elements, **SIR Table Match Rate**, which compares the SIR's selected tables to the gold SQL, and **SIR Valid but SQL Wrong Rate**, which captures cases where the intermediate symbolic plan is valid but the final SQL still produces an incorrect result.

Interpretation of Table Columns. In the results tables, the **Model** column denotes the underlying language model, the **SIR** column indicates whether an explicit symbolic intermediate representation is used, and the **Training** column denotes the training regime. A mark $\times$ indicates that no GRPO training is applied. Entries such as **1 Epoch**, **2 Epochs**, and **later epochs** refer to models from the same GRPO training run evaluated at progressively later stages of optimization.

3 Reward

We train with a shaped reward that combines symbolic quality and final SQL correctness. For each sampled completion, the total reward is defined as

$$R = R_{\text{SIR}} + R_{\text{SQL}}.$$

The symbolic component, R_{SIR}, rewards the model for producing a usable and schema-grounded intermediate representation. Specifically, the model receives positive reward when

the SIR can be parsed successfully, when it is schema-valid, and when its selected tables and columns overlap with those of the gold SQL. This part of the reward encourages the model to externalize its reasoning in a structured form before generating SQL.

The SQL component, R_{SQL}, rewards the final query at three levels. First, it rewards well-formedness and executability, namely whether the SQL parses, respects the schema, and executes successfully. Second, it rewards schema grounding in the final SQL by measuring overlap between the predicted and gold table and column sets. Third, it places the largest weight on end-task correctness: the model receives its strongest reward when the predicted SQL returns the same execution result as the gold query, together with a smaller bonus for normalized exact SQL match.

The SIR reward includes terms for successful parsing, schema validity, table-level overlap, and column-level overlap. The SQL reward includes analogous terms for SQL parsing, schema validity, successful execution, table overlap, and column overlap, together with a large execution-equivalence bonus and a smaller exact-match bonus. In our implementation, the execution-equivalence term has the highest weight, ensuring that the policy is primarily optimized for correct final answers rather than only for well-formed intermediate structures.

This reward design is particularly important for GRPO. A reward based only on final execution correctness would be sparse and difficult to optimize, especially early in training. By contrast, the symbolic and schema-level components provide denser intermediate feedback. As a result, the model is encouraged to learn a progression from valid symbolic reasoning, to valid SQL generation, to correct database execution.

Reward Terms. For reference, the symbolic reward includes four components: SIR parseability, SIR schema validity, SIR table overlap with the gold SQL, and SIR column overlap with the gold SQL. The SQL reward includes SQL parseability, SQL schema validity, SQL executability, SQL table overlap, SQL column overlap, execution-result equivalence with the gold SQL, and normalized exact SQL match. We compute table and column overlap using F_1 against the corresponding gold schema elements.

4 Results

Model	Best of N	SIR	Training	Execution Accuracy	Execution Success Rate	SQL Schema Valid Rate
Qwen3-4B	N=8	×	×	66.05	96.42	94.78
Qwen3-4B	N=1	×	×	55.80	88.49	90.23
Qwen3-4B	N=1	✓	×	58.61	97.10	95.94
Qwen3-4B	N=1	✓	1 Epoch	67.70	96.23	95.36
Qwen3-4B	N=1	✓	2 Epochs	71.57	98.16	97.87
Qwen3-4B	N=1	✓	3 Epochs	72.73	97.49	98.55
Qwen3-4B	N=1	✓	4 Epochs	72.82	97.68	98.45

Table 1: Main performance on Spider validation. Best of N indicates that N candidate SQL queries are sampled at inference time and the best candidate is selected according to execution-based accuracy. Execution Accuracy measures final answer correctness, Execution Success Rate measures whether the generated SQL executes without error, and SQL Schema Valid Rate measures whether the generated SQL is schema-adherent. Metrics are reported as percentages.

Main Performance. Table 1 reports the main results on Spider validation. The single-sample base model achieves 55.80% execution accuracy. A stronger Best-of-N zero-shot baseline improves this to **66.05**%, showing that multi-sample decoding alone already provides strong performance. Introducing SIR without training improves execution success and schema validity, indicating that the symbolic intermediate representation helps the model produce more well-formed and schema-adherent SQL even before optimization.

GRPO training further improves performance. After 1 Epoch, execution accuracy reaches **67.70**%. After 2 Epochs, 3 Epochs, and 4 Epochs, execution accuracy increases to **71.57**%,

Model	Best of N	SIR	Training	Schema-Oriented Table Match Rate	Schema-Oriented Column Exact Rate	SIR Parse OK Rate	SIR Valid Rate	SIR Table Match Rate	SIR Valid but SQL Wrong Rate
Qwen3-4B	N=8	×	×	59.96	32.21	–	–	–	–
Qwen3-4B	N=1	×	×	52.03	28.53	–	–	–	—
Qwen3-4B	N=1	✓	×	43.13	24.27	99.61	100.00	43.04	41.39
Qwen3-4B	N=1	✓	1 Epoch	58.70	35.78	99.90	100.00	57.93	32.30
Qwen3-4B	N=1	✓	2 Epochs	61.99	38.10	100.00	100.00	56.77	28.43
Qwen3-4B	N=1	✓	3 Epochs	66.34	40.43	99.81	100.00	59.28	27.27
Qwen3-4B	N=1	✓	4 Epochs	65.86	40.04	100.00	100.00	60.35	27.18

Table 2: Neuro-symbolic analysis on Spider validation. Best of N indicates that N candidate outputs are sampled at inference time and the best candidate is selected according to execution-based correctness. Schema-Oriented Table Match Rate and Schema-Oriented Column Exact Rate measure SQL-level schema grounding against the gold query. SIR Parse OK Rate, SIR Valid Rate, and SIR Table Match Rate measure the quality of the symbolic intermediate representation, while SIR Valid but SQL Wrong Rate quantifies cases where symbolic grounding is correct but final SQL realization still fails. All metrics are reported as percentages.

72.73%, and **72.82**%, respectively. SQL schema validity also remains consistently strong, reaching a peak of **98.55**% at 3 Epochs and remaining high at 98.45% after 4 Epochs. Overall, the trained SIR models achieve the best performance, but the gains across later epochs are relatively modest.

Neuro-Symbolic Analysis. Table 2 shows that the gains in final performance are accompanied by stronger schema grounding and improved symbolic reasoning. The untrained SIR model already produces highly stable symbolic outputs, with an SIR parse rate of 99.61% and an SIR validity rate of **100.00**%, but its symbolic grounding remains weak. GRPO training improves this substantially. Schema-oriented table match increases from 43.13% in the untrained SIR model to 61.99% after 2 Epochs, peaks at **66.34**% after 3 Epochs, and remains strong at 65.86% after 4 Epochs. Schema-oriented column exact rate similarly improves from 24.27% to 38.10%, **40.43**%, and 40.04% across 2, 3, and 4 Epochs. The rate of cases where the SIR is valid but the final SQL is still wrong decreases from 41.39% in the untrained model to 28.43%, 27.27%, and finally **27.18**% over the same stages. These results show that training improves both the quality of the symbolic plan and the model's ability to convert that plan into correct SQL, although the improvements beyond the earlier epochs are again relatively small.

Comparison to Best-of-N. The Best-of-N baseline is an important comparison because it shows that multi-sample decoding alone can already deliver good performance. By generating multiple candidates at inference time and selecting the best one according to execution-based correctness, Best-of-N provides a meaningful boost in execution accuracy, execution success, and schema validity. This indicates that access to multiple candidate generations is itself a valuable advantage for text-to-SQL.

The GRPO-trained SIR models can be viewed as building on top of this multi-sample benefit. GRPO also operates with multiple rollouts per prompt, but it uses them not only for selection, but also for reward-based policy improvement. In this sense, the trained models benefit from the same diversity advantage as Best-of-N, while additionally learning from symbolic validity, schema adherence, and execution-based rewards. The results therefore suggest that reinforcement learning is able to leverage the benefits of multiple sampled outputs and further improve performance through reward-guided optimization over multiple epochs.

5 Discussion

Our results suggest that the proposed method improves text-to-SQL performance in two complementary ways. First, introducing an explicit symbolic intermediate representation improves schema adherence and execution robustness, even without any training. Second, GRPO training strengthens this symbolic stage and improves the mapping from symbolic plans to final SQL. The trained models do not simply produce more valid intermediate

structures. They also become better at aligning those structures with the correct schema elements and converting them into correct executable queries.

At the same time, the results reveal an important limitation. Even in the strongest model, a non-trivial fraction of examples still produce valid SIRs but incorrect final SQL. This indicates that symbolic grounding and SQL realization are related but distinct challenges. In other words, producing a valid symbolic plan is not sufficient by itself. The model must also learn how to faithfully translate that plan into the correct final query.

Another important observation is that performance continues to improve across epochs, but the gains are not large in later stages of training. The transition from 2 Epochs to 4 Epochs yields only modest improvements in execution accuracy and symbolic grounding. This suggests that a better training strategy may be needed to make later training more effective. One promising direction is a **curriculum-based reward schedule** that places greater emphasis on SIR validity and schema grounding early in training, and gradually increases the importance of execution correctness in later stages.

More broadly, the comparison against Best-of-N highlights the value of learning over inference-time search alone. While multi-sample decoding can improve performance, GRPO training produces further gains that are accompanied by stronger schema grounding and more interpretable intermediate reasoning. This supports the central claim of the paper: explicit symbolic intermediate representations can make small language models both more accurate and more schema-aware in text-to-SQL.

6 Conclusion

We presented a neuro-symbolic approach to text-to-SQL that introduces a Schema-Grounded Symbolic Intermediate Representation between the natural language question and the final SQL query. This intermediate structure improves schema adherence, execution robustness, and overall text-to-SQL performance, while also making the reasoning process more interpretable. Our results on Spider show that SIR is beneficial even without training, and that GRPO further strengthens both symbolic grounding and final execution accuracy.

At the same time, the results indicate that valid symbolic reasoning does not always translate into correct SQL, suggesting that symbolic planning and SQL realization remain distinct challenges. A promising direction for future work is curriculum-based reward design, in which early training emphasizes symbolic validity and schema grounding, while later training places greater weight on final execution correctness.

7 Appendix

7.1 Reward Formulas

$$R = R_{\text{SIR}} + R_{\text{SQL}}$$

$$R_{\text{SIR}} = 20\,\mathbf{1}[\text{parse}] + 40\,\mathbf{1}[\text{valid}] + 60\,F_1(T^{\text{sir}}_{\text{pred}}, T_{\text{gold}}) + 40\,F_1(C^{\text{sir}}_{\text{pred}}, C_{\text{gold}})$$

SIR parse $\rightarrow$ reward if the model produces a readable symbolic plan
SIR valid $\rightarrow$ reward if the symbolic plan respects the schema
$F_1(T^{\text{sir}}_{\text{pred}}, T_{\text{gold}})$ $\rightarrow$ reward for choosing the correct tables
$F_1(C^{\text{sir}}_{\text{pred}}, C_{\text{gold}})$ $\rightarrow$ reward for choosing the correct columns

$$\begin{aligned} R_{\text{SQL}} = {} & \underbrace{20\,\mathbf{1}[\text{parse}] + 40\,\mathbf{1}[\text{schema}] + 40\,\mathbf{1}[\text{exec}]}_{\text{well-formed \& executable}} \\ & + \underbrace{60\,F_1(T^{\text{sql}}_{\text{pred}}, T_{\text{gold}}) + 40\,F_1(C^{\text{sql}}_{\text{pred}}, C_{\text{gold}})}_{\text{schema grounding}} \\ & + \underbrace{700\,\mathbf{1}[\text{Exec}(\hat{y}) = \text{Exec}(y)] + 100\,\mathbf{1}[\text{NormSQL}(\hat{y}) = \text{NormSQL}(y)]}_{\text{final correctness}}\,. \end{aligned}$$

SQL parse / schema / exec $\rightarrow$ reward valid, schema-adherent, executable SQL
$F_1(T^{\text{sql}}_{\text{pred}}, T_{\text{gold}})$, $F_1(C^{\text{sql}}_{\text{pred}}, C_{\text{gold}})$ $\rightarrow$ reward correct schema grounding in the final SQL
$\text{Exec}(\hat{y}) = \text{Exec}(y)$ $\rightarrow$ largest reward for getting the final answer correct
$\text{NormSQL}(\hat{y}) = \text{NormSQL}(y)$ $\rightarrow$ extra reward if the predicted SQL matches the gold SQL form

$\mathbf{1}[\cdot]$ = indicator function, F_1 = overlap with gold schema elements.

7.2 GRPO

We optimize the model using Group Relative Policy Optimization (GRPO). For each input, we sample a group of G candidates $\{y_i\}_{i=1}^{G}$ and compute rewards $\{r_i\}_{i=1}^{G}$. We compute a relative advantage by normalizing within the group:

$$A_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}, \quad \mu_r = \frac{1}{G}\sum_{j=1}^{G} r_j, \quad \sigma_r^2 = \frac{1}{G}\sum_{j=1}^{G}(r_j - \mu_r)^2. \tag{1}$$

We update the policy using a KL-regularized objective:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_{i\sim\{1,\ldots,G\}}[A_i \log \pi_\theta(y_i \mid x, y_0)] + \beta\,\text{KL}(\pi_\theta(\cdot \mid x, y_0) \,\|\, \pi_{\text{ref}}(\cdot \mid x, y_0))\,. \tag{2}$$

where π_{ref} is a reference policy and β controls KL regularization.

7.3 Metric Definitions

- **Execution Accuracy:** Percentage of examples where the predicted SQL returns the *same final answer* as the gold SQL.
- **Execution Success Rate:** Percentage of examples where the predicted SQL *runs without error*.
- **SQL Schema Valid Rate:** Percentage of examples where the predicted SQL uses only *valid tables and columns* from the schema.
- **Schema-Oriented Table Match Rate:** Percentage of examples where the predicted SQL selects the *same tables* as the gold SQL.
- **Schema-Oriented Column Exact Rate:** Percentage of examples where the predicted SQL selects the *same columns* as the gold SQL.

- **SIR Parse OK Rate:** Percentage of examples where the model produces a *well-formed SIR* that can be parsed successfully.
- **SIR Valid Rate:** Percentage of examples where the SIR uses only *valid schema elements*.
- **SIR Table Match Rate:** Percentage of examples where the SIR identifies the *same tables* as the gold SQL.
- **SIR Valid but SQL Wrong Rate:** Percentage of examples where the SIR is valid, but the final SQL still produces the *wrong answer*.

7.4 Example of the working pipeline

7.4.1 Example 1:

Question

List the name and date the battle that has lost the ship named 'Lettice' and the ship named 'HMS Atalanta'

Database Schema

```
Database ID: battle_death
Schema:
- battle(id, name, date, bulgarian_commander, latin_commander, result)
- ship(lost_in_battle, id, name, tonnage, ship_type, location, disposition_of_ship)
- death(caused_by_ship_id, id, note, killed, injured)
Foreign keys:
- ship.lost_in_battle = battle.id
- death.caused_by_ship_id = ship.id
```

Base Non SIR Predicted SQL

```
SELECT battle.name, battle.date
FROM battle
JOIN ship ON battle.id = ship.lost_in_battle
WHERE ship.name = 'HMS Atalanta' OR ship.name = 'Lettice';
```

Base Non SIR Output

```
[(Siege of Constantinople, 1235), (Siege of Constantinople, 1235)]
```

Trained SIR Output

```
{
  "task_type": "select",
  "tables": [
    "battle",
    "death",
    "ship"
  ],
  "joins": [
    {
      "left": "battle.id",
      "right": "ship.lost_in_battle"
    },
    {
      "left": "death.caused_by_ship_id",
      "right": "ship.id"
    }
  ],
  "select": [
    {
      "type": "column",
      "column": "name",
      "agg": null,
      "alias": null,
      "distinct": false
    },
    {
      "type": "column",
```

```
      "column": "date",
      "agg": null,
      "alias": null,
      "distinct": false
    }
  ],
  "filters": [
    {
      "column": "name",
      "op": "=",
      "value": "Lettice",
      "value_type": "literal"
    },
    {
      "column": "name",
      "op": "=",
      "value": "HMS Atalanta",
      "value_type": "literal"
    }
  ],
  "group_by": [],
  "having": [],
  "order_by": [],
  "limit": null,
  "set_op": null,
  "subquery": null,
  "notes": "The task requires listing the name and date of battles where ships named
            'Lettice' and 'HMS Atalanta' were lost. The 'ship' table is joined with
            the 'battle' table on 'lost_in_battle', and the 'death' table is joined
            on 'caused_by_ship_id'. Filters are applied to select ships with the
            specified names."
}
```


Trained SIR Predicted SQL

```
SELECT battle.name, battle.date
FROM battle
JOIN ship ON battle.id = ship.lost_in_battle
WHERE ship.name = 'Lettice'
UNION
SELECT battle.name, battle.date
FROM battle
JOIN ship ON battle.id = ship.lost_in_battle
WHERE ship.name = 'HMS Atalanta';
```

Trained SIR SQL Output

```
[(Siege of Constantinople, 1235)]
```

Correct SQL

```
SELECT T1.name , T1.date FROM battle AS T1 JOIN ship AS T2 ON T1.id = T2.lost_in_battle WHERE
T2.name = 'Lettice' INTERSECT SELECT T1.name , T1.date FROM battle AS T1 JOIN ship AS T2 ON T1.id =
T2.lost_in_battle WHERE T2.name = 'HMS Atalanta'
```

Correct Output

```
[(Siege of Constantinople, 1235)]
```

7.4.2 Example 2:

Question

How many cars have a larger accelerate than the car with the largest horsepower?

Database Schema

```
Database ID: car_1
Schema:
- continents(ContId, Continent)
```

```
- countries(CountryId, CountryName, Continent)
- car_makers(Id, Maker, FullName, Country)
- model_list(ModelId, Maker, Model)
- car_names(MakeId, Model, Make)
- cars_data(Id, MPG, Cylinders, Edispl, Horsepower, Weight, Accelerate, Year)
Foreign keys:
- countries.Continent = continents.ContId
- car_makers.Country = countries.CountryId
- model_list.Maker = car_makers.Id
- car_names.Model = model_list.Model
- cars_data.Id = car_names.MakeId
```

Base Non SIR Predicted SQL

```
SELECT COUNT(*) FROM cars_data WHERE Accelerate > (SELECT Horsepower FROM cars_data ORDER BY Horsepower DESC LIMIT 1)
```

Base Non SIR Output

```
[(0)]
```

Trained SIR Output

```
{
  "task_type": "aggregation",
  "tables": [
    "cars_data"
  ],
  "joins": [],
  "select": [],
  "filters": [
    {
      "column": "Accelerate",
      "op": ">",
      "value": "(SELECT Accelerate FROM cars_data ORDER BY Horsepower DESC LIMIT 1)",
      "value_type": "subquery"
    }
  ],
  "group_by": [],
  "having": [],
  "order_by": [],
  "limit": 1,
  "set_op": null,
  "subquery": "SELECT Accelerate FROM cars_data ORDER BY Horsepower DESC LIMIT 1",
  "notes": "The task requires counting cars with an accelerate value greater than
            the car with the largest horsepower. A subquery is used to find the
            maximum horsepower and compare it with the accelerate values."
}
```

Trained SIR Predicted SQL

```
SELECT COUNT(*) FROM cars_data WHERE Accelerate > (SELECT Accelerate FROM cars_data ORDER BY Horsepower DESC LIMIT 1);
```

Trained SIR SQL Output

```
[(39)]
```

Correct SQL

```
SELECT COUNT(*) FROM CARS_DATA WHERE Accelerate > ( SELECT Accelerate FROM CARS_DATA ORDER BY Horsepower DESC LIMIT 1 );
```

Correct Output

```
[(39)]
```

7.4.3 Example 3:

Question

Find the weight of the youngest dog.

Database Schema

```
Database ID: pets_1
Schema:
- Student(StuID, LName, Fname, Age, Sex, Major, Advisor, city_code)
- Has_Pet(StuID, PetID)
- Pets(PetID, PetType, pet_age, weight)
Foreign keys:
- Has_Pet.StuID = Student.StuID
- Has_Pet.PetID = Pets.PetID
```

Base Non SIR Predicted SQL

```
SELECT Pets.weight
FROM Pets
JOIN Has_Pet ON Pets.PetID = Has_Pet.PetID
JOIN Student ON Has_Pet.StuID = Student.StuID
WHERE Student.Age = (SELECT MIN(Age) FROM Student) AND Pets.PetType = 'dog';
```

Base Non SIR Output

```
[]
```

Trained SIR Output


```
{
  "task_type": "aggregation",
  "tables": [
    "Has_Pet",
    "Pets",
    "Student"
  ],
  "joins": [
    {
      "left": "Has_Pet.PetID",
      "right": "Pets.PetID"
    }
  ],
  "select": [
    {
      "type": "column",
      "column": "weight",
      "agg": null,
      "alias": null,
      "distinct": false
    }
  ],
  "filters": [
    {
      "column": "PetType",
      "op": "=",
      "value": "dog",
      "value_type": "literal"
    }
  ],
  "group_by": [],
  "having": [],
  "order_by": [
    {
      "column": "weight",
      "direction": "ASC",
      "agg": null
    }
  ],
  "limit": 1,
  "set_op": null,
  "subquery": null,
  "notes": "Find the weight of the youngest dog by filtering for dogs in the Pets
```

```
            table and ordering by weight in ascending order, then limiting to the
            first result."
}
```

Trained SIR Predicted SQL

```
SELECT Pets.weight
FROM Has_Pet
JOIN Pets ON Has_Pet.PetID = Pets.PetID
WHERE Pets.PetType = 'dog'
ORDER BY Pets.weight ASC
LIMIT 1;
```

Trained SIR SQL Output

```
[(9.3)]
```

Correct SQL

```
SELECT weight FROM pets ORDER BY pet_age LIMIT 1
```

Correct Output

```
[(9.3)]
```

References

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1425. URL https://aclanthology.org/D18-1425/.